R Notebook

Code

This is an R Markdown (http://rmarkdown.rstudio.com) Notebook. When you execute code within the notebook, the results appear beneath the code.

Try executing this chunk by clicking the *Run* button within the chunk or by placing your cursor inside it and pressing *Ctrl+Shift+Enter*.

charge library

if (!require('neo4r')) { # client neo4j install.packages('neo4r') }

```
library(neo4r)
neodb = neo4j_api$new(
  url = "http://localhost:7474",
  user = "neo4j",
  password = "omybioinfo"
)
cypher = function(query, neo4j=neodb, ...) call_neo4j(query=query, con=neo4j, ...)
```

1.3.2.1 Nettoyage

avant de creer la bd surtout pas apres...

```
'MATCH ()-[r]-() DELETE(r)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH ()-[r]-() RETURN count(r)' %>% cypher %>% unlist
```

```
count(r).value
            0
```

```
# attention supprime tout
'MATCH (n) DELETE(n)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n) RETURN count(n)' %>% cypher %>% unlist
```

```
count(n).value
            0
```

Genes 1.1.1.1 Choix des identifiants de référence, import genes

extraire position des gene sur ecoli

```
library(tidyverse)
library(jsonlite)
```

1ere chose a faire cest crere les noeuds

```
ncbi <- read_tsv("../data/mapping.bed.tsv", show_col_types = F)
ncbi %>% head
```

```
names(ncbi)
```

```
[1] "bnumber" "rank"    "strand"  "begin"
[5] "end"
```

Nombre de lignes attendu : 4403

```
dim(ncbi) # 4315 ok
```

```
[1] 4315    5
```

```
length(unique(ncbi$bnumber))
```

```
[1] 4305
```

Load CSV GENOME DE REF E.COLI (GENES)

```
# IMPORT

"LOAD CSV WITH HEADERS FROM 'file:///mapping.bed.tsv' AS row FIELDTERMINATOR '\t'
CREATE (n:Gene)
SET n = row,
 n.id = row.bnumber,
 n.organism = toInteger('511145'),
 n.rank = toInteger(row.rank),
 n.strand = row.strand,
 n.begin = toInteger(row.begin),
 n.end = toInteger(row.end)
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:Gene) RETURN count(n)' %>% cypher %>% unlist #4315
```

```
count(n).value
        4315
```

VISU

```
"MATCH (n:Gene) RETURN n LIMIT 6" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'CREATE INDEX ON :Gene(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=3, name='index_817871c2', type='GENERAL BTREE', schema=(:Gene {id}), indexProvider='native-btree-1.0' )'."
```

1.3.2.2.2 Keywords et liens Keyword → Gene (uniprot)

CHARGE NOEUD KEYWORDS

```
# CHECK
keywords %>% select(keyword) %>% unique
```

```
# Load CSV
keywords %>%
  select(keyword) %>%
  unique %>%
  write_csv("../neo4j.import/uniprot.keywords.csv")

'LOAD CSV WITH HEADERS FROM "file:///uniprot.keywords.csv" AS row
CREATE (n:Keyword)
SET n = row,
 n.id = row.keyword
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :Keyword(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=4, name='index_9193b500', type='GENERAL BTREE', schema=(:Key
word {id}), indexProvider='native-btree-1.0' )'."
```

VERIF

```
'MATCH (n:Keyword) RETURN count(n)' %>% cypher %>% unlist #385
```

```
count(n).value
          385
```

Liens Keyword → Gene

```
keywords %>% write_csv("../neo4j.import/uniprot.keywords.genes.csv")
```

```
"MATCH (:Keyword)-[r:describes]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.keywords.genes.csv' AS line
MATCH (k:Keyword),(g:Gene)
WHERE k.id=line.keyword AND g.id=line.bnumber
WITH k,g
MERGE (k)-[:describes]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

VERIF

```
"MATCH (:Keyword)-[r:describes]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #27323
```

```
count(r).value
        27323
```

1.3.2.2.3 PubMed

Create files

./scripts/gbff.to.bnumber.GeneID.py downloads/GCF_000005845.2.gbff.gz > generated.data/mapping.bnumber_ncbi.tsv

```
pmids = read_tsv("../data/bnumber.PMID.tsv", col_types = "cc")
pmids %>% select(PMID) %>% unique %>% write_csv("../neo4j.import/ncbi.pmid.csv")
pmids %>% rename(gene_id = bnumber) %>% write_csv("../neo4j.import/ncbi.pmid.genes.csv")
```

Export dans Noe4j

```
# LOAD dans neo4j
"MATCH (:PubMed)-[r:cites]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:PubMed) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'LOAD CSV WITH HEADERS FROM "file:///ncbi.pmid.csv" AS row
CREATE (n:PubMed)
SET n = row,
 n.id = row.PMID
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :PubMed(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=5, name='index_66c3ecb5', type='GENERAL BTREE', schema=(:Pub
Med {id}), indexProvider='native-btree-1.0' )'."
```

Vérification

```
'MATCH (n:PubMed) RETURN count(n)' %>% cypher %>% unlist #35320
```

```
count(n).value
        35320
```

Import du fichier CSV LIEN ARETE

```
"MATCH (:PubMed)-[r:cites]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///ncbi.pmid.genes.csv' AS line
MATCH (p:PubMed),(g:Gene)
WHERE p.id=line.PMID AND g.id=line.gene_id
WITH p,g
MERGE (p)-[:cites]->(g)
" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (:PubMed)-[r:cites]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #101841
```

```
count(r).value
        101841
```

Complétez cette partie afin de charger dans la base de données :

Les domaines InterPro -> Ok
les pathways,
les unités de transcription (TUs),

interpro

Load CSV NODES

```
interpro %>%
  select(keyword) %>%
  unique %>%
  write_csv("../neo4j.import/uniprot.interpro.csv")

# Check clean the bd
"MATCH (:InterPro)-[r]-() DELETE r" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:InterPro) DELETE n" %>% cypher
```

No data returned.

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
#nom de la BD
'LOAD CSV WITH HEADERS FROM "file:///uniprot.interpro.csv" AS row
CREATE (n:InterPro)
SET n = row,
 n.id = row.keyword
'  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# 6,760
```

verif

```
'MATCH (n:InterPro) RETURN count(n)' %>% cypher %>% unlist #6760 expected 6679
```

```
count(n).value
         6760
```

VERIF

```
"MATCH (n:InterPro) RETURN n LIMIT 6" %>% cypher %>% .$n
```

INDEX

```
'CREATE INDEX ON :InterPro' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Statement.SyntaxError"

$error_message
[1] "Invalid input ': expected \"(\" (line 1, column 26 (offset: 25))\n\"CREATE INDEX ON :InterPro\"\n                         ^
"
```

RECUP lien arete

```
interpro %>% write_csv("../neo4j.import/uniprot.interpro.genes.csv")
```

```
#LOAD DATA ON NEO4J
"MATCH (:InterPro)-[r:encoded_in]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.interpro.genes.csv' AS line
MATCH (k:InterPro),(g:Gene)
WHERE k.id=line.keyword AND g.id=line.bnumber
WITH k,g
MERGE (k)-[:encoded_in]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:InterPro)-[r:encoded_in]->(:Gene) RETURN count(r)" %>% cypher %>% unlist # 15401 15515
```

```
count(r).value
        15515
```

GO

Load CSV NODES

```
"MATCH (:GOTerm)-[r]-() DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH (n:GOTerm) DELETE n" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.nodes.tsv' AS row FIELDTERMINATOR '\t'
CREATE (n:GOTerm)
SET n.id = row.id,
n.name = row.desc,
n.desc  = row.def,
n.namespace = row.namespace
"  %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Verif

```
"MATCH (n:GOTerm) RETURN count(n)" %>% cypher %>% unlist # 42887
```

```
count(n).value
        42887
```

```
# 42887
```

```
"CREATE INDEX ON :GOTerm(id)" %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=6, name='index_1a03f871', type='GENERAL BTREE', schema=(:GO
Term {id}), indexProvider='native-btree-1.0' )'."
```

expected /## $error_code /## [1] “Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists” ## /## $error_message /## [1] “An equivalent index already exists, ‘Index(id=9, name=’index_1a03f871’, type=‘GENERAL BTREE’, schema=(:GOTerm {id}), indexProvider=‘native-btree-1.0’)’.” Extait du contenu :

```
"MATCH (n:GOTerm) RETURN n LIMIT 6" %>% cypher %>% .$n
```

```
Column 2 ['namespace'] of item 2 appears in position 1 in item 1. Set use.names=TRUE to match by column name, or
use.names=FALSE to ignore column names. use.names='check' (default from v1.12.2) emits this message and procee
ds as if use.names=FALSE for  backwards compatibility. See news item 5 in v1.12.2 for options to control this messag
e.
```

RECUP lien arete relation is a

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.is_a.edges.tsv' AS line FIELDTERMINATOR '\t'
MATCH (t1:GOTerm),(t2:GOTerm)
WHERE t1.id=line.term1 AND t2.id=line.term2
WITH t1,t2
MERGE (t2)-[r:is_a]->(t1)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH ()-[r:is_a]->() RETURN count(r)" %>% cypher %>% unlist #68521
```

```
count(r).value
        68521
```

RECUP lien arete relation part of

```
"
LOAD CSV WITH HEADERS FROM 'file:///go.part_of.edges.tsv' AS line FIELDTERMINATOR '\t'
MATCH (t1:GOTerm),(t2:GOTerm)
WHERE t1.id=line.term1 AND t2.id=line.term2
WITH t1,t2
MERGE (t2)-[r:part_of]->(t1)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"MATCH ()-[r:part_of]->() RETURN count(r)" %>% cypher %>% unlist #6785
```

```
count(r).value
        6785
```

```
GOTerms = uniprot %>%
  select(uniprotID=Entry, GOTerm=`Gene Ontology IDs`) %>%
  right_join(mapping, by = join_by(uniprotID)) %>% # right join to remove those without bnumber
  separate_rows(GOTerm, sep='; ') %>%
  select(bnumber, GOTerm) %>%
  arrange(bnumber)
GOTerms
```

Save CSV file for Neo4J

```
GOTerms %>% write_csv("../neo4j.import/uniprot.GOTerm.bnumber.csv")
```

Create conexion Merge into graph

```
"
LOAD CSV WITH HEADERS FROM 'file:///uniprot.GOTerm.bnumber.csv' AS line
MATCH (t:GOTerm),(g:Gene)
WHERE t.id=line.GOTerm AND g.id=line.bnumber
WITH t,g
MERGE (t)-[:annotates]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH ()-[r:annotates]->() RETURN count(r)" %>% cypher %>% unlist #22373
```

```
count(r).value
        22373
```

TUs

Charge noeud TU (list unique TU/geneTU)-> 6665

```
names(tus_to_export)
```

```
[1] "ecocyID" "gene"    "bnumber" "Right"
[5] "Left"    "Strand"  "desc"
```

```
dim(tus_to_export)
```

```
[1] 6545    7
```

```
length(unique(tus_to_export$ecocyID))
```

```
[1] 3634
```

```
# LOAD dans neo4j liste noeud 5TU unique et toutes les info relatives a chaque Tu)
"MATCH (:TU)-[r:harbors]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:TU) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"LOAD CSV WITH HEADERS FROM 'file:///ecocyc.tu.csv' AS row
CREATE (n:TU)
SET n=row,
 n.id = row.ecocyID,
 n.strand = row.Strand,
 n.begin = toInteger(row.Left),
 n.end = toInteger(row.Right),
 n.desc  = row.desc,
 n.organism = toInteger('511145')
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:TU) RETURN count(n)' %>% cypher %>% unlist #3634 expected 3696
```

```
count(n).value
        3634
```

VISU

```
"MATCH (n:TU) RETURN n LIMIT 6" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'DROP INDEX index_1e15df2a'%>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'CREATE INDEX ON :TU(id)' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Creation LIEN ARETE relation connection association

```
"MATCH (:TU)-[r:harbors]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# EXPORT
"
LOAD CSV WITH HEADERS FROM 'file:///ecocyc.tu.genes.csv' AS line
MATCH (p:TU),(g:Gene)
WHERE p.id=line.ecocyID AND g.id=line.bnumber
WITH p,g
MERGE (p)-[:harbors]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (n:TU)-[r:harbors]->(:Gene) RETURN count(r)" %>% cypher %>% unlist # expected 6841
```

```
count(r).value
         6170
```

Pathway

Charge noeud pathways (list unique pathways/gene)-> 6665

```
names(pathways_to_export)
```

```
[1] "ecocyID" "gene"    "desc"    "bnumber"
```

Export to Neo4J

```
# LOAD dans neo4j
"MATCH (:Pathway)-[r:requires]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
'MATCH (n:Pathway) DELETE n' %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
"LOAD CSV WITH HEADERS FROM 'file:///ecocyc.pathways.csv' AS row
CREATE (n:Pathway)
SET n=row,
 n.id = row.ecocyID,
 n.organism = toInteger('511145'),
 n.desc  = row.desc
 " %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

verif

```
'MATCH (n:Pathway) RETURN count(n)' %>% cypher %>% unlist #438 ->OK expected 442
```

```
count(n).value
          438
```

VISU

```
"MATCH (n:Pathway) RETURN n LIMIT 15" %>% cypher %>% .$n
```

Index sinon l’ajout de liens et les requêtes peuvent être très longs

```
'CREATE INDEX ON :Pathway(id)' %>% cypher
```

```
$error_code
[1] "Neo.ClientError.Schema.EquivalentSchemaRuleAlreadyExists"

$error_message
[1] "An equivalent index already exists, 'Index( id=8, name='index_bec32f20', type='GENERAL BTREE', schema=(:Path
way {id}), indexProvider='native-btree-1.0' )'."
```

Creation LIEN ARETE relation connection association

```
"MATCH (:Pathway)-[r:requires]->(:Gene) DELETE r" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

```
# EXPORT
"
LOAD CSV WITH HEADERS FROM 'file:///ecocyc.pathways.genes.csv' AS line
MATCH (p:Pathway),(g:Gene)
WHERE p.id=line.ecocyID AND g.id=line.bnumber
WITH p,g
MERGE (p)-[:requires]->(g)
" %>% cypher
```

```
No data returned.
```

```
list()
attr(,"class")
[1] "neo"  "neo"  "list"
```

Vérification

```
"MATCH (:Pathway)-[r:requires]->(:Gene) RETURN count(r)" %>% cypher %>% unlist #38194
```

```
count(r).value
        2747
```